

System Design Cheatsheet (End-to-End)

1. The Journey: Simple → Scalable

Client → Server → Load Balancer → CDN → Cache → Sharded/Replicated DB → Message Queue → Blob Storage *(Each component was added to fix ONE specific bottleneck.)*

2. Basic Request Flow

- 1. Client requests `example.com`
 - 2. **DNS resolver** converts URL → IP address
 - 3. Client opens **TCP connection**, wrapped in **HTTP/HTTPS**
 - 4. Server responds
-

3. Vertical vs Horizontal Scaling

Type	What it means	Limit
Vertical	Upgrade one server (more RAM/CPU)	Hardware ceiling + single point of failure
Horizontal	Add more servers	Needs a way to route traffic → Load Balancer

4. Load Balancer (LB)

Distribution algorithms:

Algorithm	How it works	Best for
Round Robin	Rotates requests evenly	Uniform request load
Least Connections	Sends to server with fewest active connections	Non-uniform request times
IP Hashing	Same client IP → same server (sticky session)	Stateful sessions

Layer 4 vs Layer 7:

	Layer 4	Layer 7
Works on	IP + TCP port	URL path, headers, cookies
Speed	Fast, no inspection	Smarter, content-aware
Example	Basic routing	AWS ALB, Nginx (static→CDN, API→app)

- LB does **health checks** → removes unhealthy servers automatically
- LB itself can fail → run **multiple LBs** (active-active / active-passive)

5. CDN (Content Delivery Network)

- Fixes: high latency for global users
- Caches **static content only** (images, JS, CSS, fonts, static HTML) — NOT dynamic API data
- **Cache hit** → edge node serves directly | **Cache miss** → fetch from origin, cache it, then serve

Cache invalidation:

- **TTL** — expires after set time
- **Versioned filenames** — e.g. `main.a2b2c3.js` forces refresh on deploy

6. Redis Caching (Dynamic Data)

- Fixes: DB read bottleneck. DB (ms) → Cache (ns)

Strategy	How	Trade-off
Write-through	Write to DB + cache together	Consistent, slower writes
Write-back	Write to cache, flush to DB later	Fast, risk of data loss

Eviction policies: LRU (least recently used) · LFU (least frequently used) · TTL (expiry)

Known problems + fixes:

- **Cache stampede** (mass miss on expiry) → **mutex lock**, only one request refetches
- **Cold start** (empty cache floods DB) → **cache pre-warming**

7. Database Scaling

Replication:

Role	Job
Primary (master)	Handles all writes
Read replicas	Handle reads, offload traffic

- Primary dies → replica promoted via **leader election**
- Trade-off: **replication lag** → stale reads on replicas (route fresh writes to primary temporarily)

Sharding (when even replication isn't enough):

Strategy	How
Range-based	e.g. user IDs A-M in shard 1, N-Z in shard 2
Hash-based	<code>hash(user_id) % num_shards</code>
Consistent hashing	Virtual ring, minimizes data movement on node change/add

→ Combine **sharding** + **replication**: each shard gets its own primary + read replicas.

8. Message Queues (Async / Event-Driven)

- Fixes: slow synchronous tasks (image resize, notifications) blocking user response

Flow: Client uploads → server responds immediately → job pushed to queue → workers process in background

Benefits: fast response · fault-tolerant (resume on crash) · easy to add new consumers (e.g. analytics)

Tool	Best for
Kafka	Persistent log, event streaming, analytics, offset tracking
RabbitMQ	General-purpose message broker

9. Blob Storage (Files/Media)

- Don't store images/videos/PDFs in relational DB — too costly/slow
 - Use **S3 / GCS / Azure Blob** — DB stores only the **URL**, not the file
 - **Pre-signed URL**: temporary signed link, lets client upload/download directly to blob storage (bypasses app server), has expiry + signature
 - Serve blobs via **CDN** for low latency
-

10. Real-Time Communication

Method	Direction	Use Case
Short polling	Client repeatedly asks	Simple, wasteful
Long polling	Connection held open till response	Lower overhead than short polling
WebSocket	Bidirectional, persistent	Chat, gaming, collaboration
SSE (Server-Sent Events)	Server → client only	Notifications, live feeds

11. Idempotency

- Prevents duplicate processing on retries (e.g. double payment)
 - Client sends **idempotency key (UUID)** → server detects repeats → same result every time
 - Used by: Stripe, PayPal
-

12. CAP Theorem

Pick 2 of 3 — **Partition Tolerance is non-negotiable** in distributed systems, so really it's **CP vs AP**:

	Guarantees	Example
CP	Consistency + Partition Tolerance	Financial systems (must be correct)
AP	Availability + Partition Tolerance	YouTube view counters (stale OK, uptime matters)

13. Full Mental Model (Recap Order)

1. Client-server + DNS
2. Horizontal scaling + **L7 Load Balancer**
3. **CDN** for static assets
4. **Redis** for dynamic data caching
5. **Sharding + Replication** for DB scale
6. **Message Queue** for async/background jobs
7. **Blob storage + pre-signed URLs** for media, served via CDN
8. Layer in: consistent hashing, idempotency, CAP trade-offs, real-time protocol choice

Golden rule: every component exists to solve one specific bottleneck — know *why* it's there, not just *that* it's there.